

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 09 · LECTURE NOTES

Delaunay Triangulation and Meshing

The triangulation you want, and how to force it to behave.

MODULE OUTCOMES

State the empty-circumcircle property and the max-min angle property.

Implement incremental Delaunay with edge flipping.

Explain constrained Delaunay and when it is needed.

Apply Ruppert refinement for quality meshing.

State what fails in three dimensions.

1 What makes it the right triangulation

FOUR EQUIVALENT CHARACTERISATIONS

Empty circumcircle: a triangulation is Delaunay when no triangle's circumcircle contains any other site in its interior. This is the definition that becomes the incircle predicate.

Max-min angle: among all possible triangulations of the same point set, the Delaunay triangulation maximises the minimum angle — and in fact lexicographically maximises the whole sorted angle vector.

This is the reason anyone wants it.

Dual of the Voronoi diagram: connect two sites exactly when their Voronoi cells share an edge (Module 08 §1).

Lifted lower hull: project the lower convex hull of the points lifted onto the paraboloid (Module 08 §3).

All four coincide when the points are in general position, and all four fail to determine a unique answer when four points are cocircular — which is the one degeneracy this module must handle.

MAX-MIN ANGLE IS THE PROPERTY THAT MATTERS, AND WHAT IT DOES NOT PROMISE

Thin slivers are bad for everything downstream. Barycentric interpolation across them is ill-conditioned; finite-element stiffness matrices built on them have terrible condition numbers; normals and gradients computed from them are numerically meaningless; and texture mapping across them stretches visibly.

Delaunay maximises the minimum angle over all triangulations of the same point set. It is provably optimal in that specific and useful sense, which is a strong statement and is why it is the default choice everywhere.

But it does not guarantee that the minimum angle is good. If the input points admit only bad triangles — four points in a very flat configuration, say — then Delaunay hands you the least bad of a set of bad options. **Optimal among the available is not the same as adequate.**

Obtaining an actual quality guarantee requires adding points, which changes the problem from 'triangulate these points' to 'mesh this domain'. That is refinement, and it is §3.

2 Construction

Start with a bounding triangle containing all sites.

```
for each site p:                                # SHUFFLE the order first
    t = triangle containing p                    # point location
    split t at p into 3 triangles
    LEGALISE each new edge

LEGALISE(edge ab, point p):
    let (p,a,b) and (a,b,d) share ab
    if incircle(p, a, b, d) > 0:                 # d inside -> edge is ILLEGAL
        flip ab to pd
        LEGALISE(ad, p); LEGALISE(db, p)        # recurse outward
```

Expected $O(n \log n)$ under RANDOM insertion order.
 $O(n^2)$ if you insert in sorted order -- so shuffle.

Shuffling the insertion order is the difference between $O(n \log n)$ and $O(n^2)$, and it is routinely omitted. The bad case is not adversarial or rare: **sorted input is extremely common**, because points frequently arrive from a scan, a grid traversal, or a previous sorted pass. Randomised incremental construction only has its expected bound if the randomisation actually happens.

WHY FLIPPING TERMINATES

Consider the vector of all angles in the current triangulation, sorted in ascending order. This is a finite vector of real numbers, and triangulations of a fixed point set can be compared by it lexicographically.

Every legalising flip increases that vector lexicographically. The flip replaces the worse of the two possible diagonals of a convex quadrilateral with the better one, and the six angles involved strictly improve in the lexicographic sense.

There are finitely many triangulations of a finite point set, so a strictly increasing process over them cannot cycle and must terminate.

And it terminates exactly at the Delaunay triangulation, because a triangulation in which no edge is illegal satisfies the empty-circumcircle property globally. **Local optimality implies global optimality here** — which is unusual, is not true of most optimisation problems, and is precisely what makes a purely local flipping procedure correct.

3 Constraints and quality

CONSTRAINED DELAUNAY: WHEN AN EDGE MUST BE PRESENT

A polygon boundary, a road, a crease, a river, the edge of a hole — these must appear in the triangulation, and the plain Delaunay triangulation of the vertices may simply not contain them.

The constrained Delaunay triangulation forces them in and then makes everything else as Delaunay as possible: a triangle's circumcircle is permitted to contain a site, but only when a constraint edge blocks visibility between them. It is the Delaunay triangulation of the visibility-restricted problem.

This is the clean way to triangulate a polygon with holes (Module 05 §4) — insert the boundary and hole edges as constraints, triangulate, and discard the triangles outside. **No bridging, and none of the bridging degeneracies.**

Conforming Delaunay is the alternative: rather than relaxing the property, split the constraint edges by inserting points until the pieces appear in the ordinary Delaunay triangulation. **The result is genuinely Delaunay and has more vertices**, which matters when the mesh is an input to simulation rather than a rendering artifact.

- 1 **Find a triangle whose smallest angle is below the target, or which is too large, and insert its circumcentre.** The circumcentre is, by the empty-circumcircle property, far from every existing vertex — which is why this does not produce new bad triangles near it.
- 2 **If the proposed circumcentre would land inside the diametral circle of a constraint segment, split that segment at its midpoint instead, and do not insert the circumcentre. This single rule is the whole subtlety of Ruppert's algorithm** and the reason it terminates.
- 3 **Repeat until no triangle is bad.** The algorithm terminates with every angle above roughly 20.7° , and with a triangle count within a constant factor of the optimal mesh for that quality bound.
- 4 **It fails when the input itself contains angles below about 60° between two constraint segments. This is fundamental, not a gap in the algorithm** — no amount of point insertion can improve the angle in the corner where two constraints meet sharply, because that angle is part of the specified domain.
- 5 **Chew's variants and Shewchuk's terminator algorithm** improve the achievable angle bound and handle small input angles by grading the mesh into them rather than attempting to fix them.

Guaranteed-quality meshing is a solved problem in two dimensions and an open one in three, which is the single most useful thing to know about this area.

4 Three dimensions

Property	Two dimensions	Three dimensions
Size of the triangulation	$O(n)$, always.	$O(n^2)$ in the worst case — points on the moment curve achieve it. A tetrahedralisation can be quadratically larger than its input.

Property	Two dimensions	Three dimensions
Max-min angle optimality	Delaunay is optimal among all triangulations.	False. There is no such guarantee in 3D, and this is the most important single fact in the table — the property that justifies using Delaunay at all simply does not transfer.
Slivers	Excluded by the empty-circumcircle property.	Permitted. Four nearly-cocircular points form a near-zero-volume tetrahedron that satisfies the empty-circumsphere property perfectly. See below.
Flipping	Always converges to Delaunay.	Can get stuck. The 2-to-3 and 3-to-2 flips are not always applicable, so a purely local flipping procedure may reach a non-Delaunay configuration with no legal move.
Polyhedra	Every simple polygon can be triangulated (Module 05 §1).	False in 3D. The Schönhardt polyhedron — a triangular prism with its top face twisted — cannot be decomposed into tetrahedra without adding interior points , which destroys the entire induction of Module 05.
Quality meshing	Solved: Ruppert refinement with proven bounds.	Open. Heuristics, optimisation, and sliver exudation are used in practice, with no guarantee.

SLIVERS: THE 3D FAILURE WITH NO 2D ANALOGUE

A **sliver** is a tetrahedron whose four vertices lie almost exactly on a common circle — so it is nearly flat, has near-zero volume, and yet **its circumsphere can be perfectly empty**.

So the Delaunay property does not exclude slivers. The criterion that guarantees good triangles in two dimensions guarantees nothing whatsoever in three, and a correctly computed 3D Delaunay tetrahedralisation of perfectly reasonable points can be full of them.

And slivers are destructive. They ruin finite-element conditioning, make interpolated gradients and normals meaningless, and cause time-step restrictions in explicit simulation (CSCE 649) that can be orders of magnitude worse than the rest of the mesh requires.

Sliver removal is a separate post-process — vertex perturbation, weight assignment (sliver exudation via a power diagram, Module 08 §4), or direct optimisation. **None of these comes with a clean guarantee, which is precisely why three-dimensional meshing remains an active research area while the two-dimensional problem has been settled for thirty years.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Shewchuk — Triangle, and Delaunay Refinement Algorithms for Triangular Mesh Generation (free)	The reference for §3 , with working code. Shewchuk's writing on meshing is unusually clear.
de Berg et al. — Computational Geometry, chapter 9 (Delaunay triangulations)	Incremental construction, the flipping termination argument, and the randomisation analysis of §2.
Shewchuk — What Is a Good Linear Finite Element? (free)	Why angle quality matters , derived from the interpolation error rather than asserted. The best answer to 'why not just use any triangulation'.
Cheng, Dey & Shewchuk — Delaunay Mesh Generation	The three-dimensional material of §4, including slivers and the Schönhardt obstruction. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement incremental Delaunay with flipping and exact incircle.
- 2 **Verify the four characterisations agree** on the same point set — compare against your Voronoi dual and your lifted hull.
- 3 Insert in sorted order and in shuffled order. **Measure the difference and plot it against n .**
- 4 Instrument the flip count per insertion and plot the distribution.
- 5 Compare the minimum angle of Delaunay against several random triangulations of the same points.
- 6 **Run on cocircular points** and report which triangulation you get and whether it is stable under reordering.
- 7 Implement constrained Delaunay and use it to triangulate a polygon with three holes.
- 8 **Implement Ruppert refinement** and plot the minimum angle against the number of inserted points.
- 9 Construct an input with a 20° corner and show that refinement does not terminate at a high angle bound.
- 10 Build a 3D Delaunay tetrahedralisation and **count the slivers**. Report the worst dihedral angle.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give four equivalent definitions of the Delaunay triangulation.
- 2 What does max-min angle guarantee, and what does it not?
- 3 Describe incremental insertion with legalisation.
- 4 Prove that flipping terminates.

- 5 Why does insertion order matter, and what should you do?
- 6 What is constrained Delaunay, and how does it differ from conforming?
- 7 Describe Ruppert refinement and its one subtle rule.
- 8 Why does refinement fail on small input angles?
- 9 Name six things that break in three dimensions.
- 10 What is a sliver and why does Delaunay not exclude it?